

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

ANALISIS KOMPLEKSITAS ALGORITMA

Bing O notation, time & Space complexity



Nama : Muhammad Alfarisi

NIM : 2025573010051

Kelas : TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN
INFORMASI DAN KOMPUTER POLITEKNIK NEGERI
LHOKSEUMAWE

2026

DAFTAR ISI

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA ANALISIS KOMPLEKSITAS	
ALGORITMA Big O notation, time & Space complexity.....	1
DAFTAR ISI	2
BAB I PENDAHULUAN.....	3
1. Latar Belakang.....	3
2. Identifikasi Masalah	3
3. Rumusan Masalah.....	3
BAB II LANDASAN TEORI.....	4
1. Pengertian Analisis Kompleksitas Algoritma	4
2. Pengertian big O notation	4
3. Pengertian time complexity.....	4
4. Struktur Analisis Kompleksitas Algoritma	4
5. Elemen Analisis Kompleksitas Algoritma	4
6. Atribut Analisis Kompleksitas Algoritma.....	5
7. Fungsi Analisis Kompleksitas Algoritma	5
BAB III PROSES PRAKTIKUM	6
3.1. Alat Dan Bahan.....	6
3.2. Proses Praktikum.....	6
BAB IV PENUTUP	16
4.1. kesimpulan	16

BAB I

PENDAHULUAN

1. Latar Belakang

berakar pada kebutuhan untuk mengukur efisiensi dan kinerja suatu algoritma secara teoritis, bukan sekadar berdasarkan kecepatan perangkat keras. Dalam pengembangan perangkat lunak, seringkali terdapat banyak algoritma yang bisa menyelesaikan satu masalah yang sama, namun dengan tingkat efisiensi yang berbeda.

2. Identifikasi Masalah

- Menentukan Efisiensi Waktu (Time Complexity)
- Menentukan Efisiensi Ruang (Space Complexity)
- Evaluasi Kinerja Pada Berbagai Kasus (Best, Average, Worst Case)
- Pemilihan Struktur Data Yang Tepat
- Analisis Kompleksitas Struktural
- Masalah Komputasi yang sulit (intractable Problems)

3. Rumusan Masalah

- Dekriptif dan konsep: Apa yang dimaksud dengan analisis kompleksitas waktu dan kompleksitas ruang pada suatu algoritma?
- Pengukuran Efisiensi: Bagaimana cara mengukur jumlah tahapan komputasi (operasi dasar) yang dilakukan oleh suatu algoritma?
- Analisis Kasus: Bagaimana menentukan kompleksitas waktu algoritma untuk kasus terburuk (worst-case), rata-rata (average-case), dan terbaik (best-case)?

BAB II

LANDASAN TEORI

1. Pengertian Analisis Kompleksitas Algoritma

Kompleksitas metode untuk mengukur efisiensi algoritma dalam hal waktu eksekusi (waktu) dan penggunaan memori (ruang) saat menangani ukuran input data (N) yang besar. Tujuannya adalah memilih algoritma terbaik yang paling hemat sumber daya, bukan berdasarkan kecepatan komputer, melainkan laju pertumbuhan operasi.

2. Pengertian big O notation

Notasi Big O adalah metode matematika untuk mengukur kompleksitas waktu (runtime) atau ruang (memori) suatu algoritma, yang menggambarkan bagaimana kebutuhan sumber daya meningkat seiring bertambahnya jumlah input (n). Ini berfokus pada batas atas (kasus terburuk) pertumbuhan fungsi.

3. Pengertian time complexity

Mengukur seberapa cepat runtime algoritma bertambah seiring peningkatan ukuran input, sementara space complexity mengukur jumlah memori tambahan yang dibutuhkan algoritma.

4. Struktur Analisis Kompleksitas Algoritma

- Jenis Kompleksitas Algoritma
- Skenario Analisis (Waktu)
- Alat Ukur (Notasi Asimptotik)

5. Elemen Analisis Kompleksitas Algoritma

- Ukuran Masuk (n)
- Kompleksitas Waktu ($T(n)$)
- Kompleksitas Ruang ($S(n)$)

Analisis Kompleksitas Algoritma

6. Atribut Analisa Kompleksitas Algoritma

- Kompleksitas Waktu (Time Complexity)
- Kompleksitas Ruang (Space Complexity)
- Ukuran Masukan (input Size-n)

7. Fungsi Analisis Kompleksitas Algoritma

Fungsi utama analisis kompleksitas algoritma adalah untuk mengukur tingkat efisiensi, kecepatan, dan penggunaan memori suatu algoritma tanpa bergantung pada perangkat keras atau bahasa pemrograman tertentu.

BAB III

PROSES PRAKTIKUM

3.1. Alat Dan Bahan

- Laptop
- Visual Studio Code
- Git
- GitHub
- Node JS.

3.2. Proses Praktikum

Langkah 1 Membuat folder Praktikum-5 di dalam repository.

```
Mkdir praktikum-5  
Cd praktikum-5
```

Langkah 2 Inisialisasi npm project

```
npm init -y
```

Analisis Kompleksitas Algoritma

Langkah 3 Membuat folder Untuk Tugas

```
// Pastikan sedang berada pada repository  
mkdir tugas
```

Langkah 4 Membuat file untuk isi folder tugas

```
Cd Praktikum-5  
touch tugas-1.js tugas-2.js
```

Langkah 5 Isi file tugas-1.js dengan code seperti code dibawah

```
// Fungsi A: Intersection dari dua array - versi  $O(n^2)$  dan  
//  $O(n)$  menggunakan Set  
//  $O(n^2)$  time,  $O(\min(n,m))$  space - nested loop  
function intersectionNaive(arr1, arr2) {  
    const result = [];  
    for (let i = 0; i < arr1.length; i++) {  
        for (let j = 0; j < arr2.length; j++) {  
            if (arr1[i] === arr2[j] &&  
!result.includes(arr1[i])) {  
                result.push(arr1[i]);  
            }  
        }  
    }  
    return result;  
}  
  
//  $O(n)$  time,  $O(\min(n,m))$  space - menggunakan Set  
function intersectionSet(arr1, arr2) {
```

Analisis Kompleksitas Algoritma

```
const set1 = new Set(arr1);
const result = [];
for (const item of arr2) {
    if (set1.has(item) && !result.includes(item)) {
        result.push(item);
    }
}
return result;
}

// Fungsi B: Kelompok anagram dari array string
// O(n * k log k) time, O(n * k) space - di mana n panjang
// array, k panjang string rata-rata
function groupAnagrams(strs) {
    const map = {};
    for (const str of strs) {
        const sorted = str.split('').sort().join('');
        if (!map[sorted]) {
            map[sorted] = [];
        }
        map[sorted].push(str);
    }
    return Object.values(map);
}

// Fungsi C: Cek jika ada dua elemen yang jumlahnya sama
// dengan kuadrat elemen ketiga
// O(n³) time, O(1) space - triple nested loop
function checkSumSquareNaive(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (i !== k && j !== k && arr[i] + arr[j]
=== arr[k] * arr[k]) {
                    return true;
                }
            }
        }
    }
}
```



```
        return false;
    }

    // O(n log n) time, O(n) space - sort array, lalu two
    // pointers atau set
    function checkSumSquareOptimized(arr) {
        const sorted = [...arr].sort((a, b) => a - b);
        const squares = new Set(sorted.map(x => x * x));
        for (let i = 0; i < sorted.length; i++) {
            for (let j = i + 1; j < sorted.length; j++) {
                const sum = sorted[i] + sorted[j];
                if (squares.has(sum)) {
                    return true;
                }
            }
        }
        return false;
    }

    // Fungsi untuk mengukur waktu eksekusi
    function measureTime(label, fn, ...args) {
        const start = Date.now();
        const result = fn(...args);
        const end = Date.now();
        console.log(`${label}: ${end - start} ms`);
        return result;
    }

    // Data besar untuk testing
    const arr1 = Array.from({ length: 1000 }, () =>
        Math.floor(Math.random() * 1000));
    const arr2 = Array.from({ length: 1000 }, () =>
        Math.floor(Math.random() * 1000));
    const strs = Array.from({ length: 1000 }, () => {
        const chars = 'abcdefghijklmnopqrstuvwxyz';
        let str = '';
        for (let i = 0; i < 5; i++) {
            str += chars[Math.floor(Math.random() *
                chars.length)];
        }
    });
```

Analisis Kompleksitas Algoritma

```
    }
    return str;
});
const nums = Array.from({ length: 100 }, () =>
Math.floor(Math.random() * 50) + 1); // Lebih kecil untuk
O(n³) agar tidak terlalu lama

console.log('=== Analisis dan Benchmark Fungsi ===');

// Fungsi A
console.log('\n--- Fungsi A: Intersection ---');
const naiveResult = measureTime('Intersection Naive
O(n²)', intersectionNaive, arr1, arr2);
const setResult = measureTime('Intersection Set O(n)',
intersectionSet, arr1, arr2);
console.log('Hasil sama?',
JSON.stringify(naiveResult.sort()) ===
JSON.stringify(setResult.sort()));

// Fungsi B
console.log('\n--- Fungsi B: Group Anagrams ---');
const anagramResult = measureTime('Group Anagrams O(n k
log k)', groupAnagrams, strs);
console.log('Jumlah kelompok:', anagramResult.length);

// Fungsi C
console.log('\n--- Fungsi C: Check Sum Square ---');
const naiveCheck = measureTime('Check Sum Square Naive
O(n³)', checkSumSquareNaive, nums);
const optCheck = measureTime('Check Sum Square Optimized
O(n log n)', checkSumSquareOptimized, nums);
console.log('Hasil sama?', naiveCheck === optCheck);
```

Analisis Kompleksitas Algoritma

Hasil Run terminal untuk file tugas-1.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-
$ node tugas-1.js
=== Analisis dan Benchmark Fungsi ===

--- Fungsi A: Intersection ---
Intersection Naive  $O(n^2)$ : 9 ms
Intersection Set  $O(n)$ : 0 ms
Hasil sama? true

--- Fungsi B: Group Anagrams ---
Group Anagrams  $O(n \cdot k \cdot \log k)$ : 5 ms
Jumlah kelompok: 995

--- Fungsi C: Check Sum Square ---
Check Sum Square Naive  $O(n^3)$ : 1 ms
Check Sum Square Optimized  $O(n \log n)$ : 1 ms
Hasil sama? true
```

Langkah 6 Masuk ke file tugas-2.js dan isi code seperti dibawah

```
// Fungsi untuk benchmark berbagai kompleksitas
function fn_01(n) {
    return n + 1; //  $O(1)$  - operasi konstan
}

function fn_0n(n) {
    let sum = 0;
    for (let i = 0; i < n; i++) {
        sum += i; //  $O(n)$  - loop linear
    }
    return sum;
}

function fn_OnLogn(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
```

Analisis Kompleksitas Algoritma

```
        for (let j = 1; j < n; j *= 2) {
            count++; //  $O(n \log n)$  - loop luar n, dalam
log n
        }
    }
    return count;
}

function fn_On2(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            count++; //  $O(n^2)$  - nested loop
        }
    }
    return count;
}

// Fungsi benchmarkSemua
function benchmarkSemua(ukuranData) {
    console.log('=== Benchmark Kompleksitas Algoritma
===\n');

    for (const n of ukuranData) {
        console.log(`Ukuran data: ${n}`);
        console.log('-----');

        // Ukur waktu untuk setiap fungsi
        const start01 = Date.now();
        fn_01(n);
        const time01 = Date.now() - start01;
        console.log(`O(1): ${time01} ms`);

        const start0n = Date.now();
        fn_On(n);
        const time0n = Date.now() - start0n;
        console.log(`O(n): ${time0n} ms`);

        const startOnLogn = Date.now();
```

Analisis Kompleksitas Algoritma

```
fn_OnLogn(n);
const timeOnLogn = Date.now() - startOnLogn;
console.log(`O(n log n): ${timeOnLogn} ms`);

const startOn2 = Date.now();
fn_On2(n);
const timeOn2 = Date.now() - startOn2;
console.log(`O(n²): ${timeOn2} ms`);

console.log(''); // Baris kosong
}

// Komentar observasi
console.log('=== Observasi Pola Pertumbuhan ===');
console.log('O(1): Waktu konstan, tidak berubah dengan n. Konsisten dengan teori.');
```

console.log('O(n): Waktu meningkat linear dengan n. Untuk n=100 ke 1000 (10x), waktu ~10x. Konsisten.');

console.log('O(n log n): Lebih cepat dari O(n²) tapi lebih lambat dari O(n). Untuk n besar, log n signifikan. Konsisten.');

console.log('O(n²): Waktu meningkat drastis (kuadratik). Untuk n=100 ke 1000 (10x), waktu ~100x. Konsisten, tapi tidak scalable.');

console.log('Secara keseluruhan, pola pertumbuhan konsisten dengan teori Big O: $O(1) < O(n) < O(n \log n) < O(n^2)$.');

```
}
```

// Panggil benchmarkSemua dengan ukuran data yang diberikan

```
benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Analisis Kompleksitas Algoritma

Hasil	Run tterminal untuk file tugas-2.js
-------	-------------------------------------

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-5/t
● $ node tugas-2.js
=== Benchmark Kompleksitas Algoritma ===

Ukuran data: 100
-----
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 1 ms

Ukuran data: 500
-----
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 1 ms

Ukuran data: 1000
-----
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n²): 3 ms

Ukuran data: 5000
-----
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 85 ms

Ukuran data: 10000
-----
O(1): 0 ms
```

Analisis Kompleksitas Algoritma

```
Ukuran data: 10000
-----
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n2): 296 ms

=== Observasi Pola Pertumbuhan ===
O(1): Waktu konstan, tidak berubah dengan n. Konsisten dengan teori.
O(n): Waktu meningkat linear dengan n. Untuk n=100 ke 1000 (10x), waktu ~10x. Konsisten.
O(n log n): Lebih cepat dari O(n2) tapi lebih lambat dari O(n). Untuk n besar, log n signifikan.
O(n2): Waktu meningkat drastis (kuadratik). Untuk n=100 ke 1000 (10x), waktu ~100x. Konsisten,
ble.
Secara keseluruhan, pola pertumbuhan konsisten dengan teori Big O: O(1) < O(n) < O(n log n) < O(n2)
```

ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-5/tugas <ma
\$

BAB IV

PENUTUP

4.1. kesimpulan

Big O Notation adalah metrik standar untuk mengukur efisiensi algoritma (skenario terburuk) berdasarkan pertumbuhan waktu (time) dan memori (space) saat input data (n) meningkat. Fokusnya adalah laju pertumbuhan, bukan waktu tepatnya. Ini membantu programmer memilih algoritma tercepat dan terhemat memori.